

CSCE 641 COMPUTER GRAPHICS · MODULE 05 · LECTURE NOTES

Visibility: Depth, Culling, and the Z-Buffer

Deciding what is in front, cheaply and in any order.

MODULE OUTCOMES

Implement a depth buffer and explain why it is order-independent.

Explain z-fighting quantitatively and apply the right fix.

Implement backface and frustum culling, and state what each saves.

Explain early-Z and the shader features that silently disable it.

Describe why transparency breaks the depth buffer, and the options.

1 Hidden surface removal

Several surfaces project onto the same pixel. Only the nearest opaque one should be visible. The question is how to determine which without an expensive global ordering.

1.1 Why sorting fails

The painter's algorithm sorts triangles by depth and draws them back to front. It is intuitive and it does not work. Three triangles can be arranged so that A occludes B, B occludes C, and C occludes A — a cycle, admitting no valid ordering. Interpenetrating triangles fail even more simply: neither is wholly in front. Both cases require splitting geometry at render time, which is expensive, introduces cracks, and changes your triangle count unpredictably.

1.2 The z-buffer

Catmull's 1974 solution resolves occlusion per pixel rather than per primitive. Keep a second buffer holding, for each pixel, the depth of the nearest fragment seen so far.

```
clear(depth, +INF);

for each fragment f:           // ANY order
    if (f.z < depth[f.x][f.y]) {
        depth[f.x][f.y] = f.z;
        color[f.x][f.y] = shade(f);
    }
```

THE PROPERTY THAT MATTERS

The result is **independent of submission order**. Cyclic overlap and interpenetration are handled exactly, with no special case, because the comparison happens at the only granularity where the question is well-posed: the individual sample.

The cost is a full-resolution buffer and the bandwidth to read and write it. In 1974 that was prohibitive. Today the depth buffer is simply assumed, and hardware compresses it aggressively.

2 Z-fighting

When two surfaces lie at nearly the same depth, their quantised depth values collide. Whichever fragment is processed last wins, and since processing order shifts with camera motion and draw order, the surfaces flicker against each other in a characteristic shimmering pattern.

Module 03 established the cause: stored depth is a function of $1/z$, so precision is concentrated near the camera and sparse far away. The governing quantity is the ratio f/n , not the absolute distances. A frustum with $n = 0.01$ and $f = 1000$ has a ratio of 100,000 and will z-fight on anything distant; raising n to 0.5 drops the ratio to 2,000 and usually eliminates the problem entirely.

Fix	Mechanism	Trade-off
Push the near plane out	Directly reduces f/n , the dominant term.	Geometry closer than n is clipped. Usually acceptable; try this first.
Reversed-Z with float depth	Float precision is dense near 0; reversing maps the far plane to 0, so the two non-uniformities cancel.	Requires <code>GL_GREATER</code> , zero-to-one clip control, and updating every shader that reads depth. Do it once, benefit forever.
Fix the content	Do not author coplanar surfaces where they can be avoided.	Free, and frequently the correct answer. A decal 0.5mm above a wall is a content decision, not a renderer bug.
Polygon offset	Adds a slope-scaled bias in depth to push one surface away.	Over-biasing detaches the surface visually — the same peter-panning failure mode as shadow bias in Module 10.
Depth pre-pass	Render depth only, then render colour with <code>GL_EQUAL</code> .	Doubles vertex work; pays for itself when fragment shaders are expensive, and helps overdraw as a side effect.

3 Culling: the hierarchy of not drawing

Every culling technique is the same idea applied at a different granularity: determine cheaply that something cannot contribute, and skip it. The organising principle is that coarse rejection is cheap *per unit of work avoided*, so you should reject as high up the hierarchy as you can.

Level	Granularity	Test	Typical saving
Frustum culling	Object	Bounding sphere or box against six planes	Large in open scenes; most of the world is off-screen.
Occlusion culling	Object	Bounding volume against a depth representation of what is already drawn	Very large indoors; hardest to implement well.
Backface culling	Triangle	Sign of signed screen-space area	~50% of triangles on any closed mesh, for one subtraction.
Hierarchical Z	Tile	Tile depth range against coarse depth buffer	Automatic in hardware; depends on draw order.
Early-Z	Fragment	Depth test moved before the fragment shader	Proportional to overdraw; often the biggest single win.

3.1 Backface culling

After projection, compute the signed area of the triangle in screen space — the same edge-function sum from Module 04. Its sign is the winding direction; one sign means the triangle faces away from the camera. On a closed mesh, exactly half the triangles face away at any moment, so this removes half the rasterization work for the cost of a subtraction you had already computed.

It is only valid for closed geometry. A single-sided plane, a sheet of cloth, or a leaf card must have culling disabled, or it vanishes when viewed from behind. This is the cause of the classic 'the wall disappears when I walk through it' behaviour.

3.2 Early-Z and how it is lost

Logically the depth test runs after the fragment shader, since the shader may modify depth. In practice the hardware runs it first whenever it can prove that is equivalent, which means occluded fragments are discarded before any shading work occurs. In a scene with significant overdraw this is frequently the largest single saving in the frame.

WHAT SILENTLY DISABLES EARLY-Z

Writing `gl_FragDepth` — the hardware cannot know the depth in advance.

Calling `discard` — the fragment might not write depth at all, so the depth buffer cannot be updated speculatively.

Shader side effects: image stores, atomics, SSBO writes.

The practical consequence: adding one `discard` to a foliage shader can cost far more than the `discard` itself, because it turns off early rejection for that entire draw. Mitigate with `layout(early_fragment_tests) in;` where the semantics permit it, or with a depth pre-pass.

4 Transparency

The depth buffer stores one depth per pixel because it assumes one visible surface per pixel. Transparency violates that assumption directly: a transparent surface does not replace what is behind it, it combines with it, so the final colour depends on every surface along the ray and on the order they are composited.

The standard over operator is not commutative:

$$\mathbf{C} = \alpha_s \mathbf{C}_s + (1 - \alpha_s) \mathbf{C}_a$$

Swap two layers and you get a different colour. So transparent geometry must be composited back to front — which reintroduces the painter's algorithm and all of its failure cases, now at the level of individual transparent surfaces rather than the whole scene.

4.1 The practical approach

```
// Pass 1 -- opaque, front to back (maximises early-Z rejection)
glEnable(GL_DEPTH_TEST);
glDepthMask(GL_TRUE);
glDisable(GL_BLEND);
draw(sort_front_to_back(opaque));

// Pass 2 -- transparent, back to front
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_FALSE); // test against depth, but do not write it
draw(sort_back_to_front(transparent));
```

The depth mask is the line that is most often forgotten. Transparent surfaces must still be *occluded* by opaque geometry, so the depth test stays on; but if they *write* depth they will occlude each other, and the nearest transparent surface will erase every transparent surface behind it. The symptom is transparent objects that mysteriously punch holes in one another.

4.2 When sorting is not enough

Per-object sorting fails whenever a single object overlaps itself — a glass teapot, a particle cloud, foliage. The options then are: alpha testing (order-independent but hard-edged, and it kills early-Z); weighted blended order-independent transparency (an approximation that is robust and cheap, and the pragmatic default in most engines today); depth peeling (exact, but costs a full scene pass per depth layer); or per-pixel fragment lists with atomics (exact, but needs unbounded memory and careful fallbacks).

5 You now have a renderer

Modules 02 and 03 gave you the transform chain. Module 04 gave you coverage, barycentric interpolation, and perspective correction. This module adds depth and culling. Those are, together, a complete renderer: Project 1 is now buildable end to end, with no GPU, no graphics API, and nothing in it you did not write yourself.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
GAMES101 Lecture 07 — Shading 1 (Z-buffering section)	The z-buffer and painter's algorithm comparison, including the cyclic overlap case.
NVIDIA — Depth Precision Visualized	The authoritative treatment of z-fighting and reversed-Z, with the diagrams that make the $1/z$ distribution obvious.
Fabian Giesen — 'A trip through the Graphics Pipeline', early-Z and hierarchical Z parts	How Hi-Z, early-Z, and depth compression actually work in hardware, and exactly what turns them off.
McGuire & Bavoil — Weighted Blended Order-Independent Transparency	Open-access paper. The practical OIT method, with code.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Add a z-buffer to your rasterizer. Verify order-independence by rendering the same scene with the triangle list shuffled and confirming the output images are bit-identical.
- 2 Construct the three-triangle cyclic overlap case and render it with a painter's algorithm and with the z-buffer. Capture both; the painter's version cannot be made correct by any ordering.
- 3 Implement backface culling. Measure the fragment count with and without on a closed mesh and confirm it is close to half.

- 4 Deliberately set near = 0.001 and far = 10000 and find a camera position that z-fights. Then fix it by raising the near plane, and record the ratio at which the artifact disappears.
- 5 Implement reversed-Z in your software rasterizer: it is just a different projection matrix and a flipped comparison. Compare depth precision across the view distance against the standard mapping.
- 6 **Project 1 is now due.** Assemble Modules 02–05 into the complete software rasterizer described in the syllabus, with both required screenshots.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give a configuration of triangles for which no draw order is correct, and explain why the z-buffer handles it without a special case.
- 2 Why is the f/n ratio the governing quantity for depth precision rather than the absolute far distance?
- 3 Explain how reversed-Z recovers precision, and list the four changes required to adopt it.
- 4 Name three things in a fragment shader that disable early-Z, and explain why each one does.
- 5 Why must transparent geometry disable depth writes but keep the depth test?
- 6 Per-object back-to-front sorting is insufficient for a glass teapot. Why, and what would you use instead?